

AUTONOMOUS API EXPLORER

Техническая документация проекта для Agentic Workflows Hackathon

Кратко. Агент исследует недокументированное веб-приложение через безопасный браузерный контур, связывает UI-контекст с сетевыми вызовами, формулирует и проверяет гипотезы, а затем выпускает доказательно обоснованную спецификацию API.

Версия 1.0 | 29 августа 2026 | Русская техническая документация

Основание: обсуждение концепции проекта и бриф Agentic Workflows Hackathon (стр. 1–7).

Оглавление

Номера страниц формируются автоматически при сборке документа.

Оглавление	2
1. Резюме и ценностное предложение	4
1.1. Границы MVP	4
2. Market landscape и позиционирование	5
2.1. Формула продукта	5
3. Архитектура решения	6
3.1. Контракт наблюдений	6
4. Generic browser agent: baseline, который действительно честен	7
4.1. Generic agent loop	7
4.2. Что остаётся за рамками baseline	7
5. Benchmark: контролируемая MiniCRM	8
5.1. Набор кейсов	8
6. Ground truth и canonical reconstruction schema	9
6.1. Почему смысл можно оценивать без LLM	9
6.2. Верхнеуровневая schema	9
6.3. Canonical vocabulary	9
7. Deterministic evaluator без LLM	10
7.1. Важная граница	10
7.2. Отдельные уровни качества	10
8. Benchmark runner и воспроизводимый запуск	11
8.1. Рекомендуемая структура репозитория	11
8.2. Важное разделение доступа	11
9. Финальный агент: специализированный workflow	12
9.1. Пример активного эксперимента	12
9.2. Правило документации	12
10. Fairness, metrics и VARS	13
10.1. Primary metric: VARS	13
10.2. Отчётность без выдуманных результатов	13
11. Evidence, provenance и trajectories	14
11.1. Формат provenance	14
12. Safety и human control	15
12.1. Принципы для hackathon submission	15

13. План работ до дедлайна	16
13.1. Условие остановки score	16
14. Промпты для Cursor и ожидаемые артефакты	17
14.1. Cursor prompt: MiniCRM и benchmark	17
14.2. Cursor prompt: canonical schema + evaluator	17
14.3. Cursor prompt: browser harness + baseline	17
14.4. Cursor prompt: specialized AAE	17
14.5. Ожидаемые артефакты	17
Приложение А. Чек-лист benchmark и fairness	18
Приложение В. Чек-лист evidence и safety	18
Приложение С. Чек-лист финальной подачи	18
Приложение D. Источники и решения	18

1. Резюме и ценностное предложение

Autonomous API Explorer (AAE) превращает наблюдаемое поведение веб-приложения в проверяемую документацию API. Он не «угадывает» назначение маршрутов по одному HAR-файлу: агент связывает действие в интерфейсе, HTTP-запрос, ответ и итоговое состояние UI, затем повторно проверяет важные гипотезы.

Кто	Проблема	Ценностное предложение
Разработчикам, интегрирующим legacy или внутренние веб-системы	API существует, но документации нет или она устарела; вручную сопоставлять UI, XHR и бизнес-смысл долго и ненадёжно.	OpenAPI, endpoint-документация, enum-таблицы, dependency graph, workflows и ссылки на доказательства.
Командам миграции / QA / platform engineering	Знания о скрытых параметрах, переходах статуса и зависимостях живут в интерфейсе и у отдельных людей.	Воспроизводимая карта поведения приложения с уровнем уверенности и проверяемыми основаниями.

Главный тезис: ценность создаёт не crawl сам по себе, а цикл экспериментального reverse engineering: наблюдать → выдвинуть гипотезу → безопасно проверить → зафиксировать только подтверждённое.

1.1. Границы MVP

Поддерживаемое в MVP	Не поддерживается в MVP
Single-page web apps; REST/JSON; cookie или Bearer auth; Chromium; XHR/fetch; тестовая/синтетическая среда.	GraphQL, WebSocket, gRPC-web, CAPTCHA, OAuth-исследование, SSO между доменами, загрузки файлов, платежи, произвольные production-сайты.

2. Market landscape и позиционирование

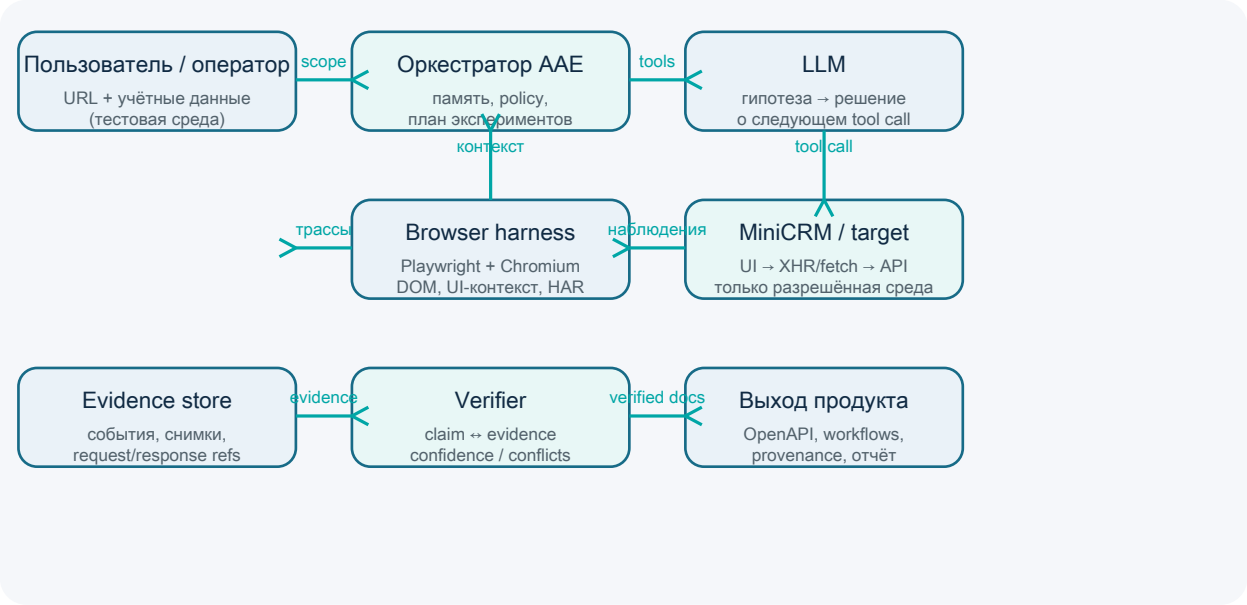
Существующие подходы полезны, но каждый закрывает только часть работы. AAE занимает промежуток между пассивным просмотром трафика и ручным reverse engineering.

Подход	Сильные стороны	Слабые стороны	Позиционирование
DevTools / HAR / прокси	Показывает фактические запросы и ответы.	Не объясняет, какое UI-действие вызвало запрос и что означают значения епит.	Использует эти артефакты как первичные доказательства.
OpenAPI scanners / traffic-to-spec	Строит синтаксический каркас API.	Обычно не восстанавливает бизнес-семантику, workflows и скрытые зависимости.	Добавляет UI-контекст, гипотезы и verification loop.
RPA / browser automation	Выполняет известные сценарии.	Не умеет самостоятельно реконструировать неизвестную модель API.	Переиспользует браузер как инструмент исследования.
Ручной reverse engineering	Даёт богатое понимание контекста.	Дорог, плохо масштабируется, трудно воспроизвести и доказать.	Формализует путь от наблюдения до доказуемого claim.
LLM по HAR-файлу	Быстро даёт черновик документации.	Слабее в семантике, зависимостях и защите от правдоподобных галлюцинаций.	Это разумный простой baseline, но не финальный агент.

2.1. Формула продукта

Вход: URL + разрешённые тестовые учётные данные. Выход: структурированная реконструкция, OpenAPI, human-readable docs, граф сущностей и workflows, confidence report и evidence bundle. Все артефакты должны ссылаться на наблюдения, а не на «веру в модель».

3. Архитектура решения



LLM не имеет прямого доступа к localhost или production-сети. Локальный runner запускает Chromium через Playwright, выполняет tool calls, собирает наблюдения и передаёт модели только разрешённый контекст. Это отделяет принятие решения от фактического управления браузером.

3.1. Контракт наблюдений

Страница	URL, видимый текст, доступные элементы и состояния форм.	Понимать, что делает пользовательское действие.
Сеть	Метод, нормализованный путь, статус, request/response body, время и correlation id.	Восстановить операцию, параметры и схемы.
Переход	Какое действие произошло до/после запроса; изменения UI.	Вывести effect и связать event с видимой семантикой.
Память	Подтверждённые claims, неопределённости, покрытие, незакрытые гипотезы.	Не повторять открытия и планировать эксперимент с наибольшей ценностью.

4. Generic browser agent: baseline, который действительно честен

Baseline — не один запрос к модели с фразой «Explore the application». Это один general-purpose LLM-agent с базовыми browser tools, одинаковым target и тем же output contract, что у финальной системы. Он может исследовать интерфейс и видеть сеть, но не получает специализированных процедур управления покрытием, памяти гипотез, активного экспериментирования или verifier-a.

SYSTEM / task

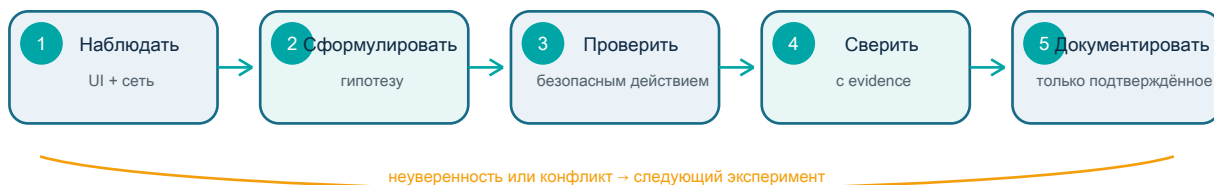
You are given access to an undocumented web application. Explore it with the provided browser tools and reconstruct as much of its HTTP API as possible.

You may inspect UI context and network events. You do not have source code, database access, API documentation or ground truth. Do not invent information. When finished, call submit_reconstruction(reconstruction).

TOOLS

observe_page() | click(element_id) | fill(element_id, value) | select(element_id, value)
go_back() | get_network_events() | submit_reconstruction(reconstruction)

4.1. Generic agent loop



Ограничения run: единый action budget, wall-clock budget и token budget. Завершение — вызов submit_reconstruction либо исчерпание бюджета. Structured tool arguments исключают ситуацию, где модель вместо результата печатает произвольный Markdown.

4.2. Что остаётся за рамками baseline

- явное планирование по information gain и coverage gap;
- долговременный журнал гипотез и конфликтов;
- обязательная повторная проверка claims;
- risk classifier и policy-компилятор действий;
- отдельный verifier / генерация доказательного отчёта.

5. Benchmark: контролируемая MiniCRM

Для объективной оценки ААЕ используется не произвольный production-сайт, а недокументированная MiniCRM: синтетическое веб-приложение с известной авторами моделью. API-документация и исходный код скрыты от агента. Он работает только через UI и разрешённые network observations.

Объект	Действия и ограничения	Сложные моменты
Customers	Поиск, создание, карточка, обновление, запрет на удаление клиента с заказами.	query/body parameters, dependency, бизнес-ограничение.
Orders	Список, карточка, создание, редактирование, статус, комментарии.	path params, enum semantics, UI-trigger, state transition.
Products	Каталог и типы product/service/bundle.	enum mapping через несколько UI-сценариев.
Payments / tasks	Связанные действия и сущности.	многошаговые workflows и ID flow.

5.1. Набор кейсов

Цель — 10+ фиксированных cases, включая трудный case. Каждый содержит отдельный seed, входную роль и ожидаемый набор ground-truth facts. Ни baseline, ни финальный агент не видят labels. Рекомендуемый первоначальный набор: 12 cases.

Кейс	Сложность	Описание
Операции и параметры	4	Список заказов: page, status, customer_id; карточка: path id.
Enum и переходы состояния	3	«Mark as shipped» → PATCH body status=4 → UI «Shipped».
Dependencies / constraints	2	DELETE customer при наличии orders → rejected.
Workflows	2	Создать customer → получить customer_id → создать order → добавить payment.
Трудный case	1	Параметр имеет UI-label, но неочевидное числовое API-значение и нужен повторный эксперимент.

6. Ground truth и canonical reconstruction schema

Ground truth — version-controlled dataset истинных, атомарных фактов о MiniCRM. Он отделён от агента и доступен только evaluator-у. Сравнение строится не по свободному prose, а по нормализованным идентификаторам и разрешённому canonical vocabulary.

6.1. Почему смысл можно оценивать без LLM

Свободный текст вроде «seems to update an order after shipment» не следует использовать как основной объект scoring: deterministic evaluator не обязан понимать, что это близко к «marks an order as shipped». Вместо этого агент подаёт machine-readable fact. Для человека описание остаётся в финальной документации, но primary score считается по фактам.

```
{
  "enum_mappings": [{
    "operation": "PATCH /api/orders/{id}",
    "field": "status",
    "value": 4,
    "meaning": "shipped",
    "confidence": 0.96,
    "evidence_ids": ["ev_044", "ev_061"]
  }],
  "operation_effects": [{
    "operation": "PATCH /api/orders/{id}",
    "field": "status",
    "value": 4,
    "ui_state": "shipped"
  }]
}
```

6.2. Верхнеуровневая schema

Category	Canonical Schema	Examples
operations	method + normalized path	GET /api/orders; PATCH /api/orders/{id}
parameters / schemas	operation + location + name + type / requiredness	query.status: integer; path.id: integer
enum_mappings	field + value + canonical meaning	order.status 4 → shipped
operation_effects / triggers	изменение + UI action	status=4; triggered_by = Mark as shipped
business_rules	condition + outcome	customer_has_orders → delete rejected
dependencies / workflows	from → artifact → to	POST customers → customer_id → POST orders
evidence	stable observation reference	request/response/UI transition/screenshot

6.3. Canonical vocabulary

Ground truth задаёт допустимые canonical labels: new, confirmed, processing, shipped, cancelled. Агенту сообщается правило использовать точный UI-visible label, приведённый к lower_snake_case. Алиасы, если нужны, объявляются заранее в открытом evaluation config; evaluator не применяет скрытую «семантическую магию».

7. Deterministic evaluator без LLM

Evaluator не «читает» смысл файла как человек. Он валидирует JSON Schema, нормализует допустимые значения, сравнивает предсказанные atomic facts с ground truth и выдаёт полный audit: matched, missing, spurious и invalid. Поэтому результат стабилен, дешёв и воспроизводим.

Шаг	Описание задачи	Результат
1. Validate	Проверить JSON Schema, типы, обязательные ключи и ссылочную целостность evidence.	Невалидный output не засчитывается; причина сохранена.
2. Normalize	Привести method, paths, IDs, whitespace и canonical labels к заданным правилам.	Сопоставимые ключи без языковой интерпретации.
3. Match	Сравнить каждому предсказанию ground-truth fact по type и canonical key.	TP / FP / FN по каждой категории.
4. Score	Посчитать precision, recall, F1, hallucination rate и VARS.	Итоговые метрики + по-case breakdown.
5. Audit	Проверить evidence references; сформировать diff.json и HTML/Markdown report.	Судья и разработчик видят, что именно зачтено.

7.1. Важная граница

Детерминированный evaluator не признаёт sent эквивалентом shipped, если такого алиаса нет в публичной таблице. Это намеренное ограничение benchmark-а: он измеряет способность восстановить наблюдаемые canonical facts, а не вкус LLM-judge. Качественный prose оценивается отдельно демонстрацией и человеком.

7.2. Отдельные уровни качества

Уровень	Как оценивается	Цель
Primary quantitative	Детерминированный fact-level evaluator.	Честное сравнение baseline и ААЕ.
Qualitative product quality	Human review готовых OpenAPI/docs/workflows.	Проверить ясность и полезность для реального пользователя.
Evidence quality	Доля supported claims, валидность provenance.	Проверить, что агент не маскирует догадки под факты.

8. Benchmark runner и воспроизводимый запуск

Runner — это не LLM и не evaluator. Он является диспетчером эксперимента: поднимает MiniCRM из чистого seed, запускает выбранную систему в изолированном браузере, применяет budget, сохраняет trajectories и передаёт submitted reconstruction evaluator-у.

Операция	Описание операции	Цели операции
Reset	Поднимает контейнер MiniCRM, seed data и роль кейса.	Независимость runs и повторяемость.
Launch	Запускает baseline или AAE с одинаковыми tool surface, версиями и бюджетами.	Fairness ресурсов.
Capture	Сохраняет последовательность tools, observations, сеть, screenshots и result JSON.	Траектория и evidence доступны для проверки.
Evaluate	Вызывает deterministic evaluator с case ground truth.	Одинаковая rubric без ручной подгонки.
Aggregate	Фиксирует seed, model version, wall time, tokens, cost и результаты.	Полный experiment ledger.

8.1. Рекомендуемая структура репозитория

```
apps/minicrm/ # target, без docs для agent
benchmarks/cases/ # seeds + public case config
benchmarks/ground-truth/ # evaluator-only facts
packages/schema/ # canonical output schema
packages/evaluator/ # deterministic scorer + reports
packages/browser-harness/ # Playwright tools + capture
agents/baseline/ # generic prompt + loop
agents/aae/ # specialist workflows
artifacts/runs/<run-id>/ # trajectories, evidence, output
docs/ # README, reproduction, changelog
```

8.2. Важное разделение доступа

Код MiniCRM и ground truth не должны попадать в tool context агента; иначе benchmark измерит утечку, а не исследование. Runner имеет к ним доступ лишь для reset и scoring. Публичны: контракт результата, список разрешённых tools, budgets и canonical vocabulary.

9. Финальный агент: специализированный workflow

Финальная система получает ту же задачу и базовый браузерный доступ, что baseline, но добавляет осмысленные механизмы, каждый из которых проверяется отдельной итерацией. Это не «больше агентов ради агентов», а набор функций, устраняющих конкретные failure modes.

Компонент	Функция	Пользовательский результат
Coverage planner	Видит непокрытые разделы, операции и неразрешённые поля.	Меньше пропущенных endpoints / paths.
Hypothesis ledger	Хранит claim, evidence, confidence, конфликт и предложенную проверку.	Не теряет контекст и не превращает догадку в факт.
Experiment planner	Выбирает безопасный UI-шаг, который различит несколько гипотез.	Лучше enum semantics и business rules.
Verifier	Требует evidence для claim и отклоняет конфликтующие/одиочные факты.	Меньше hallucinations.
Artifact generator	Конвертирует canonical JSON в OpenAPI, docs и workflows.	Готовый пользовательский результат без влияния на score.

9.1. Пример активного эксперимента

Наблюдение: на странице orders обнаружен PATCH /api/orders/{id} с body {"status":4}. Одного запроса недостаточно, чтобы назвать значение. Планировщик сохраняет гипотезу, нажимает «Mark as shipped» на другом тестовом заказе и проверяет: UI label, request body и post-state совпадают. После двух независимых наблюдений claim получает evidence-backed confidence; иначе остаётся uncertain.

9.2. Правило документации

Наблюдаемое → подтверждённое. Система может вывести полезное prose-объяснение, но утверждение в продукции должно иметь ссылку на facts/evidence или быть явно отмечено как hypothesis / low confidence.

11. Evidence, provenance и trajectories

Каждый evidence item получает стабильный ID и хранится вместе с run. Agent никогда не видит ground truth, но может ссылаться на собственные наблюдения. Evaluator проверяет, что ссылка существует и относится к заявленному run/case.

Evidence type	Representative fields	Example value in JSON
ui_action	URL, element label, action, before/after snapshot refs.	«Mark as shipped» был нажат.
network_event	method, path, status, request/response body hash/ref, correlation id.	PATCH /orders/123, body status=4.
state_transition	entity ref, before/after fields, UI state.	Processing → Shipped.
experiment	hypothesis id, decision, expected discriminant, result.	Повтор для подтверждения mapping.
policy_decision	risk class, allow/block/approval, reason.	DELETE blocked in benchmark policy.

11.1. Формат provenance

```
{
  "claim_id": "claim_enum_order_status_4",
  "canonical_fact": "order.status:4=shipped",
  "confidence": 0.96,
  "evidence_ids": ["ev_044", "ev_061"],
  "verification": {"status": "passed", "rule": "two_independent_observations"},
  "run_id": "aae-case07-seed41"
}
```

Для submission нужно включить representative trajectories каждого использованного агента: инструкции, tool call, ответ инструмента, наблюдение, следующий шаг, retry и human checkpoint. Это прямо демонстрирует agentic workflow вместо статического финального файла.

12. Safety и human control

AAE предназначен для synthetic, sandbox или явно разрешённой среды. Он не пытается «проверить всё возможное» ценой последствий. Risk policy применяется до выполнения действия и может потребовать симуляцию или человека.

Category	Operation	Policy
READ_ONLY	Открыть страницу, изменить фильтр, GET /api/*.	Разрешить автоматически.
SAFE_MUTATION	Создать тестовый draft в seeded sandbox.	Разрешить только в контролируемом case; логировать.
REVERSIBLE	Изменить тестовый статус, добавить comment.	Разрешить при allowlist; сохранить ID и rollback plan.
DESTRUCTIVE	Удалить customer, сбросить пароль, убрать пользователя.	Блокировать либо симулировать; human approval, если вообще разрешено.
EXTERNAL_SIDE_EFFECT / UNKNOWN	Отправить email, payment, publish, непонятное действие.	Блокировать. Никаких внешних эффектов без отдельного разрешения.

12.1. Принципы для hackathon submission

- Использовать synthetic или разрешённые anonymized data; не включать credentials в репозиторий.
- Показывать контроль consequential actions через sandbox/simulation и human approval.
- Отделить личные данные и секреты от artifacts; redact request bodies при необходимости.
- Связывать каждое заявление о результате с доступным evidence.
- Дать судьям путь запуска из чистого окружения.

13. План работ до дедлайна

В предоставленном брифе точная дата дедлайна не указана. Поэтому план привязан к D-0 — фактическому дню подачи. При старте 29 августа 2026 и доступных двух неделях это D-14 ... D-0; если времени меньше, сохранять этапы в том же порядке и сокращать scope, а не quality gates.

Действие	Результат работы	Критерий успеха
D-14 – D-13	Зафиксировать MVP, public contract, risk policy, репозиторий и reproducible dev environment.	Schema versioned; MiniCRM scope не расширяется.
D-12 – D-10	Собрать MiniCRM: 12 cases, seed/reset, скрытый ground truth, deterministic evaluator.	Evaluator имеет golden tests и выдаёт diff.
D-9 – D-8	Собрать browser harness и generic baseline; сохранить первые trajectories.	Baseline завершает valid submit_reconstruction на ≥10 cases.
D-7 – D-5	Добавить coverage planner, hypothesis ledger и active experiments.	Каждый компонент имеет target failure mode и ablation run.
D-4	Добавить verifier, evidence bundle и генерацию OpenAPI/docs.	Нет claims без evidence или явной uncertainty.
D-3	Прогнать baseline/final на фиксированных seeds; зафиксировать complete ledger.	Fairness checklist подписан; никакой cherry-picking cases.
D-2	Написать README, reproduction guide, improvement changelog; выбрать trajectories.	Чистое окружение воспроизводит один полный run.
D-1	Записать до 5 минут видео; проверить ссылки и artefacts.	Видео показывает problem → baseline → run → comparison → learning.
D-0	Финальный smoke test и подача.	Четыре обязательные deliverables доступны судье.

13.1. Условие остановки scope

Если к D-5 не готовы resettable MiniCRM, deterministic evaluator и baseline, остановить добавление функций агента. Без честной измеримой основы финальная система не сможет доказать improvement — это важнее визуальной сложности.

14. Промпты для Cursor и ожидаемые артефакты

Ниже — стартовые рабочие запросы. Их нужно адаптировать к фактическому стеку, но не позволять Cursor изменять benchmark contract или ground truth без review.

14.1. Cursor prompt: MiniCRM и benchmark

Создай минимальное недокументированное MiniCRM для benchmark-a Autonomous API Explorer.
Требования: REST/JSON, UI для Customers/Orders/Products, XHR/fetch, 12 фиксированных seeded cases, status enum и dependency customer→order. API docs в UI не показывать.
Отдельно реализуй reset(seed), роль case-user и скрытый ground-truth exporter, который не доступен browser agent-у. Добавь тесты reset и сценариев.
Не добавляй GraphQL, WebSocket, OAuth, платежи или внешние side effects.

14.2. Cursor prompt: canonical schema + evaluator

Реализуй versioned JSON Schema для reconstruction: operations, parameters, enum_mappings, operation_effects, triggers, business_rules, dependencies, workflows, evidence.
Напиши deterministic evaluator: schema validation → normalization → atomic-fact matching → TP/FP/FN → per-category precision/recall/F1 → VARS → diff.json. Не используй LLM, embeddings или fuzzy semantic matching.
Добавь golden tests на valid match, missing fact, spurious fact, invalid evidence_id и canonical-label mismatch.

14.3. Cursor prompt: browser harness + baseline

Собери Node.js Playwright harness для локальной MiniCRM. Экспортируй только: observe_page, click, fill, select, go_back, get_network_events, submit_reconstruction. Логируй tool calls, UI snapshots и normalised network events с correlation ids. Добавь generic LLM baseline loop без planner, persistent hypothesis memory и verifier.
Одинаковые budgets должны конфигурироваться для baseline и final agent.

14.4. Cursor prompt: specialized AAE

Добавь поверх browser harness: coverage planner, hypothesis ledger, safe experiment planner, risk classifier и evidence verifier. Каждый claim должен иметь confidence и evidence_ids. Если evidence конфликтует или недостаточно, пометь claim uncertain и не включай его в verified output. Сохраняй trajectory в artifacts/runs/<run-id>.

14.5. Ожидаемые артефакты

Артефакт	Содержимое	Критерий приемлемости
README + changelog	Проблема, пользователь, baseline, итерации, main failure mode / hot take.	Соответствует deliverable 01 hackathon brief.
Reproduction guide	Версии, setup, exact commands baseline/final/evaluation, runtime/cost.	Чистая машина воспроизводит основной результат.
Benchmark package	Cases, public config, protected ground truth, evaluator, golden tests.	Одинаковое scoring для обеих систем.
Run artifacts	Trajectory, evidence, reconstruction JSON, reports, aggregate ledger.	Каждый итоговый claim проверяем.
Demo video ≤5 min	Problem, baseline, realistic run, comparison, changelog, removed experiment.	Понятно без устного пояснения.

Приложение А. Чек-лист benchmark и fairness

Критерий	Проверено
☐ Public output schema и canonical vocabulary одинаковы для baseline и final agent.	
☐ Cases, seeds, target version, поль, tools, action/time/token budgets совпадают.	
☐ Ground truth и исходный код target недоступны в tool context агента.	
☐ Evaluator deterministic; нет LLM, embedding или скрытого fuzzy matching.	
☐ Есть ≥10 cases и отдельный challenging case.	
☐ Полный ledger содержит case-level score, runtime, cost, seed и model version.	
☐ Любое различие ресурсов объяснено в отчёте.	
☐ Результаты не подменены целевыми/примерными значениями.	

Приложение В. Чек-лист evidence и safety

Критерий	Проверено
☐ У каждого verified claim есть evidence_ids из того же run/case.	
☐ Evidence включает UI context + network event для семантических переходов.	
☐ Conflicting / one-off observation остаётся uncertainty, а не утверждением.	
☐ Destructive, external и unknown actions блокируются или симулируются.	
☐ Нет production credentials, PII или чувствительных request bodies в submission.	
☐ Human approval показан для действий, выходящих за safe sandbox policy.	

Приложение С. Чек-лист финальной подачи

Критерий	Проверено
Complete code + Improvement Changelog	☐ Чётко показано, что было до конкурса и что добавлено; есть main failure mode и hot take.
Reproduction guide	☐ Есть exact commands, data, expected output, versions, runtime и cost.
Solution video	☐ До 5 минут; есть baseline, end-to-end run, comparison, greatest contribution и removed experiment.
Agent trajectories	☐ Для каждого агента: instruction, tools, tool responses, feedback, retries и checkpoints.

Приложение D. Источники и решения

1. Agentic Workflows Hackathon, предоставленный бриф, стр. 1–7: постановка задачи, baseline, changelog, evaluation, judging, ground rules и deliverables.
2. Рабочее обсуждение концепции ААЕ: архитектура browser-agent loop, MiniCRM benchmark, canonical facts, deterministic evaluation, evidence-first documentation и safety policy.

Документ намеренно не заявляет числовых результатов до проведения реальных прогонов. Все значения VARS, improvement и cost должны быть заполнены из experiment ledger перед подачей.